



Fleet Operations: A Lambda-Architecture Data Pipeline for Ride-Hailing Utilisation and Daily Profitability

*answering a live question and a financial question from one immutable
master dataset*

A mini-project report submitted to the

Department of Electrical and Information Engineering
Faculty of Engineering
University of Ruhuna
Sri Lanka

in partial fulfilment of the requirements of

EC8203 — Applied Big Data Engineering

by

Ahamed N.F.F. - EG/2021/4390
Safeer S.M. - EG/2021/4771
Nafla M.N.P. - EG/2021/4687

Use Case 1: Ride-Hailing Fleet Operations
Academic Year 2026 | Mini-Project Report

Abstract

A ride-hailing operator needs two answers that do not fit the same processing model. A dispatcher needs to know which vehicles are idle and which zones are earning *now*, within seconds, and does not mind if one trip is missing. Finance needs to know which vehicles are becoming unprofitable once fuel and maintenance costs are applied, and that figure must be exact, complete, and restatable when a fuel partner resubmits a corrected file days later. This report presents a data platform that answers both from one immutable master dataset, and defends the architecture that makes doing so possible.

We implement a **Lambda architecture** and reject Kappa explicitly. The decisive argument is not latency but *recomputation cost*: the correction that forces a restatement does not arrive in the event stream at all. It arrives on a separate channel, as a daily CSV drop, long after the telemetry it relates to, so replaying the log — Kappa’s entire recomputation story — does not address the actual source of restatement. A date-partitioned Parquet lake lets the batch layer read exactly the one partition it needs, reconciling a full simulated day in 9.4 seconds, where Kappa would require Kafka retention measured in months to offer the same guarantee.

Lambda’s best-known weakness is two codebases that drift. Our mitigation is structural: a shared `common/` package holds the simulated clock, zone mapping, validation, haversine distance, fare maths and all seven profitability flags, and is imported by the simulators, the speed layer, the batch layer and the serving API. Two duplications exist deliberately, both to avoid a per-row Python UDF, and each is pinned to its Python counterpart by a test that feeds identical inputs through both implementations.

The system ingests a Kafka telemetry stream from a 50-vehicle simulated fleet and a daily expense file from fuel and garage partners, on a compressed clock where one real second is one simulated minute. Spark Structured Streaming archives the raw stream and computes live views; Airflow orchestrates a nine-task PySpark reconciliation per simulated day; PostgreSQL serves both layers side by side behind a FastAPI surface that labels every figure `source: speed` or `source: batch` and never blends them.

The central evidence is quantitative and, deliberately, unflattering. The speed layer runs a 10-simulated-minute watermark against injected lateness of up to 20 minutes, so the two layers are *designed* to disagree. Over four reconciled days the revenue drift was 0.000000, 0.003384, 0.000000 and 0.000000. The one non-zero day is attributable to a single late `trip_end` event dropped by the watermark and recovered by the batch layer (`speed_trips` 316 against `batch_trips` 317). We then measured *why* the drift is usually zero — 15 rows dropped per day, of which 3.4% are revenue-bearing, giving an expectation of about half an event per day — and report that arithmetic rather than inflating the fault rates to manufacture a larger number. Recomputation is demonstrated directly: a corrected v2 expense file reproduced the same 50 rows

with different values, and a forced replay returned bit-identical figures.

Observability spans all five pipeline stages: structured JSON logs carrying both wall-clock and simulated time, Prometheus metrics for arrival, progress and correctness, and ten alert rules of which five have been verified firing *and delivered* end to end, with a sixth deliberately suppressed by an inhibition rule. 202 automated tests pass, and a 28-check smoke test passes against the live stack from a fresh clone.

Six defects found by operating the system — not by reading the code — are reported in full, including a green healthcheck on an archiver that had silently stopped writing for 26 minutes, and an interval expressed in real seconds that was 60 times longer than intended in business terms.

Acknowledgements

We thank the staff of the Department of Electrical and Information Engineering, University of Ruhuna, for the module and the environment in which a project of this scope could be attempted inside two weeks, and the lecturer in charge of EC8203 for a brief that was specific enough to argue with — the tension between its two sub-questions is what made the architecture decision worth making rather than merely worth stating.

This system is assembled almost entirely from open-source work. We thank the maintainers of Apache Kafka, Apache Spark, Apache Airflow, PostgreSQL, FastAPI, Prometheus, Alertmanager, Grafana and Docker. We also thank the authors whose published arguments about Lambda and Kappa we engage with in Chapter 2; disagreeing with a well-made case is a far better way to learn an architecture than accepting one.

Contents

Abstract	i
Acknowledgements	iii
Contents	v
List of Figures	vi
List of Tables	vi
Acronyms	vii
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives and Scope	2
1.4 Contributions	3
1.5 Report Organisation	3
2 Literature Review	4
2.1 The Lambda Architecture	4
2.2 The Kappa Architecture and the Critique of Lambda	4
2.3 Event Time, Watermarks and Delivery Semantics	5
2.4 The Tooling Landscape	5
2.5 Positioning of This Work	6
3 System Architecture and Methodology	7
3.1 The Decision	7
3.2 Why Kappa Was Rejected	8
3.3 What Lambda Costs, and Our Mitigation	8
3.4 Technology Stack	9
3.5 The Simulated Clock	9
3.6 The Data Contract	9
4 Implementation	12
4.1 Ingestion	12
4.1.1 Partitioning Is a Correctness Decision	12
4.1.2 Deliberate Fault Injection	12
4.1.3 The Daily Batch Source	13
4.2 The Speed Layer	13
4.2.1 The Archiver Is Deliberately Boring	13
4.2.2 Stateful Idle Alerts	13
4.2.3 Idempotent Sinks	14
4.3 The Batch Layer and Orchestration	14
4.3.1 The Batch Layer Trusts Nothing	14
4.3.2 The Schedule Is a Poll, Not a Cron	14
4.3.3 Recomputation Is Delete-then-Insert	15
4.4 Storage and Serving	15
4.4.1 The Lambda Merge Is Explicit, Never Blended	15
4.5 Observability	15
4.5.1 What Is Measured, and Why	15

4.5.2	Structured Logging Across All Stages	15
4.5.3	Tracing — An Honest Position	16
4.5.4	Alerting Decisions Worth Defending	16
5	Results and Analysis	18
5.1	Evaluation Method	18
5.2	The Consistency Argument	18
5.2.1	Results Over Four Simulated Days	19
5.2.2	Why the Drift Is Usually Zero, and Why That Is the Stronger Finding	19
5.3	Recomputation Demonstrated	20
5.4	Fault Handling and Data Quality	20
5.5	Profitability Outcomes	20
5.6	Alerting Verification	20
5.7	Verification Summary	21
6	Discussion	22
6.1	What Running It Actually Taught Us	22
6.2	Limitations and Threats to Validity	23
6.3	At Production Scale	24
6.3.1	Ingestion	24
6.3.2	Storage	24
6.3.3	Processing	24
6.3.4	Serving	24
6.3.5	Observability	25
6.3.6	The Simulated Clock	25
7	Conclusions and Recommendations	26
7.1	Summary of Achievements	26
7.2	Key Findings	26
7.3	Limitations	26
7.4	Recommendations and Future Work	27
7.5	Lessons Learned	27
	References	29
A	Repository Layout	30
B	Database Schema	31
C	Alert Rule Inventory	32
D	API Surface	33
E	Statement of Individual Contributions	34

List of Figures

3.1	System architecture	7
5.1	Why the two layers disagree	18

List of Tables

1.1	The brief contains two sub-questions with incompatible requirements. No single processing model serves both well.	2
3.1	The two deliberate duplications, and the tests that make them defensible. Each test feeds identical inputs through both implementations and asserts identical outputs.	9
3.2	Technology selection, with the rejected alternative for each layer.	10
4.1	Injected fault rates, configured against measured, and what each fault demonstrates.	12
4.2	The metric design. Each row answers a question that the others cannot. . . .	16
5.1	Reconciliation outcomes over four consecutive simulated days. Revenue drift is the relative difference between the speed layer’s and the batch layer’s daily revenue.	19
5.2	The arithmetic of the watermark’s loss, measured over one simulated day. . .	19
5.3	Verification summary. Every row was run against the live stack.	21
A.1	Top-level layout of the submitted repository.	30
B.1	Serving-store tables, by owning layer.	31
C.1	The ten Prometheus alert rules. <i>Verified</i> means observed firing and delivered into <code>alert_notifications</code>	32
D.1	The eleven documented endpoints, by the layer that answers them.	33
E.1	Principal areas of responsibility by team member.	34

Acronyms

API	-	Application Programming Interface	RF	-	Replication Factor
CSV	-	Comma-Separated Values	RPC	-	Remote Procedure Call
CU	-	Currency Unit (simulated)	SLA	-	Service-Level Agreement
DAG	-	Directed Acyclic Graph	SLO	-	Service-Level Objective
DLQ	-	Dead-Letter Queue	SQL	-	Structured Query Language
DQ	-	Data Quality	SVG	-	Scalable Vector Graphics
ETL	-	Extract, Transform, Load	UDF	-	User-Defined Function
GPS	-	Global Positioning System	UUID	-	Universally Unique Identifier
HTTP	-	Hypertext Transfer Protocol	YAML	-	YAML Ain't Markup Language
JSON	-	JavaScript Object Notation	ZK	-	ZooKeeper
JVM	-	Java Virtual Machine			
KRaft	-	Kafka Raft (metadata mode)			

Chapter 1

Introduction

A ride-hailing operator runs two businesses at once. One is an operations business, conducted minute by minute: where are the cars, how many are earning, which zones are hot, which driver has been sitting still for an hour. The other is a financial business, conducted a day at a time: what did each vehicle earn, what did it cost to run, and is it still worth keeping on the road. This project builds a single data platform that answers both, and argues that the interesting engineering decision is not how to answer either one but how to answer both without letting the fast answer contaminate the exact one.

1.1 Background

The assessment brief [1] specifies a use case with two ingestion paths of deliberately different character. A streaming source emits GPS and telemetry events per vehicle every few seconds: `trip_id`, `driver_id`, `vehicle_id`, latitude, longitude, speed, status, fare and timestamp. A daily batch source drops one file per simulated day carrying vehicle expense records from garages and fuel partners: `vehicle_id`, `fuel_cost`, `maintenance_cost`, `distance_covered` and `service_flag`.

These two paths differ in more than cadence. The telemetry stream is high-volume, continuous, owned by us, and tolerant of small losses. The expense file is low-volume, periodic, owned by a third party, and *authoritative about money*. It can also be wrong, and be corrected: a fuel-card partner that mis-keys a reconciliation will resubmit a corrected file days later, and the figures we published in the meantime must be restated. That asymmetry, more than any latency requirement, drives the architecture in this report.

1.2 Problem Statement

The brief poses one business question:

What is fleet utilisation and earnings by area and time of day right now, and which vehicles are becoming unprofitable once yesterday's fuel and maintenance costs are factored in?

Read carefully, that is two questions whose non-functional requirements are mutually incompatible, as Table 1.1 sets out.

A dispatcher deciding where to send a car needs an answer in seconds and does not care if one trip is missing. A finance analyst deciding whether to retire a vehicle

Table 1.1: The brief contains two sub-questions with incompatible requirements. No single processing model serves both well.

	Live utilisation	Daily profitability
Latency required	seconds	next morning is acceptable
Accuracy required	approximate is fine	exact — it is money
Completeness	best effort	must cover the whole day
Recomputable?	no need	yes — partners resubmit corrected files
Consumer	dispatcher watching a map	finance, making retire/keep decisions

needs a figure that is complete, exact and *reproducible*, because the fuel-card partner can and does resubmit a corrected expense file days later. Serving both from one processing model means either making the dispatcher wait for exactness they do not need, or letting finance quote a number whose error is unbounded and unknown. The problem this project solves is how to provide both guarantees from one dataset while making it impossible for a consumer to confuse them.

1.3 Objectives and Scope

Objectives

1. Choose between a Lambda and a Kappa architecture on the evidence of this use case's latency, replay, cost and consistency requirements, and defend the rejected alternative specifically rather than dismissively.
2. Implement both ingestion paths as simulated Python sources: a continuous Kafka producer and a once-per-simulated-day file drop, with deliberate faults so that error handling can be demonstrated rather than asserted.
3. Implement processing consistent with the declared architecture, performing meaningful transformation — cleaning, zone enrichment, windowed aggregation, stateful alerting, and a join between the two sources — not pass-through.
4. Land results in a queryable store and serve a consolidated daily report and a live dashboard that answer the business question.
5. Instrument every stage with structured logging, metrics and alerting, so the pipeline's health and its *known imprecision* are both observable.
6. Quantify the cost of the architecture's approximation rather than assuming it, and report the measurement honestly whichever way it falls.

Scope and simplifications

The system runs on a single 16 GB laptop from one command (**make up**) and is demonstrable inside a lab session. Three simplifications follow from that, and each is stated where it matters rather than buried: simulated time is compressed 60-fold, so one real second is one simulated minute and a simulated day takes 24 real minutes; Kafka runs as a single broker with replication factor 1; and batch Spark runs in local mode, so the project demonstrates distributed *streaming* but not distributed *batch*. Authentication, multi-tenancy and cost-based query optimisation are out of scope. Chapter 6 sets out what each simplification would become at production scale.

1.4 Contributions

1. **An architecture decision argued from the correction channel, not from latency.** We show that the usual Lambda-versus-Kappa comparison — which turns on replay cost — resolves differently here because the event that forces recomputation never enters the event log at all (Section 3.2).
2. **A structural mitigation for Lambda’s two-codebase problem**, with the two deliberate duplications pinned together by tests that feed identical inputs through both implementations (Section 3.3).
3. **A measured, rather than asserted, account of speed-layer error.** We report a revenue drift of zero on three days of four and explain arithmetically why zero is the expected outcome, instead of inflating the injected fault rate to produce a more impressive number (Section 5.2).
4. **Six defects found by operating the system**, each generalising beyond this project, including a healthcheck that stayed green over an archiver that had stopped writing (Section 6.1).

1.5 Report Organisation

Chapter 2 reviews the Lambda and Kappa positions and the stream-processing semantics the implementation relies on. Chapter 3 states the architecture decision, the rejected alternatives, and the technology chosen for each layer with its justification. Chapter 4 describes the implementation stage by stage: ingestion, speed layer, batch layer and orchestration, storage and serving, and observability. Chapter 5 presents the measured results, with the speed-versus-batch consistency evidence as its core. Chapter 6 discusses what operating the system taught us, the limitations and threats to validity, and what would change at production scale. Chapter 7 concludes. The appendices carry the repository layout, the database schema, the alert inventory, the API surface and the statement of individual contributions.

Chapter 2

Literature Review

This chapter reviews the two architectures the brief asks us to choose between, and the stream-processing semantics that the choice depends on. We review them in order to *use* them: every claim made here is called upon in Chapter 3 or Chapter 4.

2.1 The Lambda Architecture

Marz and Warren propose Lambda as an answer to a specific problem: how to compute arbitrary functions over arbitrary data at scale, when no single system can be simultaneously fast, exact and fault-tolerant [2]. Their construction has three parts. An *immutable master dataset* records raw facts and is never updated in place. A *batch layer* recomputes views over the whole master dataset from scratch. A *speed layer* computes incremental, approximate views over only the recent data the batch layer has not yet covered. A serving layer merges the two.

Two properties of that construction matter for this project. First, **human fault tolerance**: because the master dataset is immutable and the batch views are a pure function of it, a bug in view computation is repaired by fixing the code and recomputing, not by attempting to correct derived state in place. Helland makes the more general version of this argument — that immutability converts a class of distributed-state problems into append-only bookkeeping [3]. Second, the speed layer’s error is explicitly *temporary*: it covers only the window the batch layer has not yet recomputed, and is discarded when the batch view catches up.

Kleppmann situates the same idea in a wider treatment of derived data, and makes the point this project leans on most heavily: the distinction between a system of record and a derived view is what makes recomputation safe, and losing that distinction is what makes data pipelines unmaintainable [4].

2.2 The Kappa Architecture and the Critique of Lambda

Kreps’s critique is the standard case against Lambda and deserves to be taken at full strength [5]. His argument is not that Lambda is incorrect but that it is *expensive in the wrong currency*. Maintaining two implementations of the same business logic — one in a batch framework, one in a streaming framework — doubles the code that must be written, tested and reasoned about, and, worse, produces two implementations that can silently disagree. Debugging then means debugging the divergence rather than either system.

His proposed replacement, commonly called Kappa, removes the batch layer entirely. There is one processing path, a stream processor. Recomputation is achieved not by a separate batch job but by *replaying the log* from a chosen offset into a new version of the job, and cutting over when it catches up. The log, not a file system, is the system of record.

The critique is strong and largely correct as stated. It rests, however, on an assumption worth naming explicitly, because it is the assumption this project’s use case violates: that **everything needed to recompute a result is in the log**. Where that holds, Kappa’s replay is a complete recomputation story and Lambda’s batch layer is redundant machinery. Section 3.2 argues that it does not hold here, and that the failure is structural rather than a matter of tuning.

2.3 Event Time, Watermarks and Delivery Semantics

The correctness argument in Chapter 5 depends on three results from the stream-processing literature.

Event time versus processing time. Akidau et al. formalise the distinction in the Dataflow model: an event’s *event time* is when it happened; its *processing time* is when the system saw it [6]. The two diverge arbitrarily under network delay, retries and partition skew. Any aggregation that groups by processing time produces results that depend on infrastructure behaviour rather than on the world, which for a financial figure is disqualifying.

Watermarks are a bet, not a guarantee. Because a system cannot know whether more data will arrive for a past window, it must decide when to stop waiting. A watermark is that decision: a heuristic assertion that no event older than some bound will now arrive. Dataflow and its predecessor MillWheel both make the same point — the watermark trades completeness against latency, and events later than the watermark are *dropped*, not deferred [6, 7]. This is the mechanism we measure directly in Section 5.2: our watermark is deliberately set tighter than the injected lateness, so the loss is real and observable rather than hypothetical.

End-to-end exactly-once requires idempotent sinks. Spark’s Structured Streaming provides exactly-once semantics with respect to its own state and offsets, through checkpointing and write-ahead logging [8], building on the micro-batch fault-tolerance model of discretised streams [9] and the lineage-based recovery of RDDs [10]. That guarantee stops at the sink boundary: an arbitrary external sink written from `foreachBatch` observes *at-least-once* delivery, because a failure between writing and committing offsets causes the batch to be re-executed. Kleppmann’s treatment is the clearest statement of the remedy — make the write idempotent, typically by keying it, so a repeated delivery is indistinguishable from a single one [4]. Every sink in this project does that, and Section 4.2.3 gives the keys.

2.4 The Tooling Landscape

Kafka’s original design paper sets out the properties we rely on: a partitioned, replicated, append-only commit log, with ordering guaranteed *within* a partition and not across them [11, 12]. That single sentence is the reason partitioning is a correctness decision in this project rather than a throughput one (Section 4.1.1).

For stream processing, the principal alternative to Spark Structured Streaming is Apache Flink, whose design treats streaming as primary and batch as a bounded special case, with a stronger state model [13]. We acknowledge in Section 3.4 that Flink is arguably the better stream processor considered in isolation, and explain why an engine that also runs our batch layer wins on a criterion Flink cannot help with.

2.5 Positioning of This Work

The literature gives a clear decision rule that is, in practice, rarely applied: choose Lambda when recomputation cannot be expressed as a log replay, and choose Kappa when it can. Most published comparisons instead weigh operational simplicity against latency, which is a tie the fashionable option usually wins.

This project contributes a worked case where the rule bites. The correcting event — a partner’s revised expense file — arrives on a separate channel, in a different format, days after the telemetry it restates. It is not late data in the watermark sense; it is not in the log at all. Furthermore, where most treatments assert that the speed layer is approximate, we *measure* the approximation on a system configured so that the two layers must disagree, and report the result including the days on which the disagreement was zero.

Chapter 3

System Architecture and Methodology

3.1 The Decision

We implement a **Lambda architecture**: an immutable master dataset written by a dedicated archiver, a *speed layer* computing approximate live views, and a *batch layer* recomputing exact views from the complete archive. Figure 3.1 shows the resulting system.

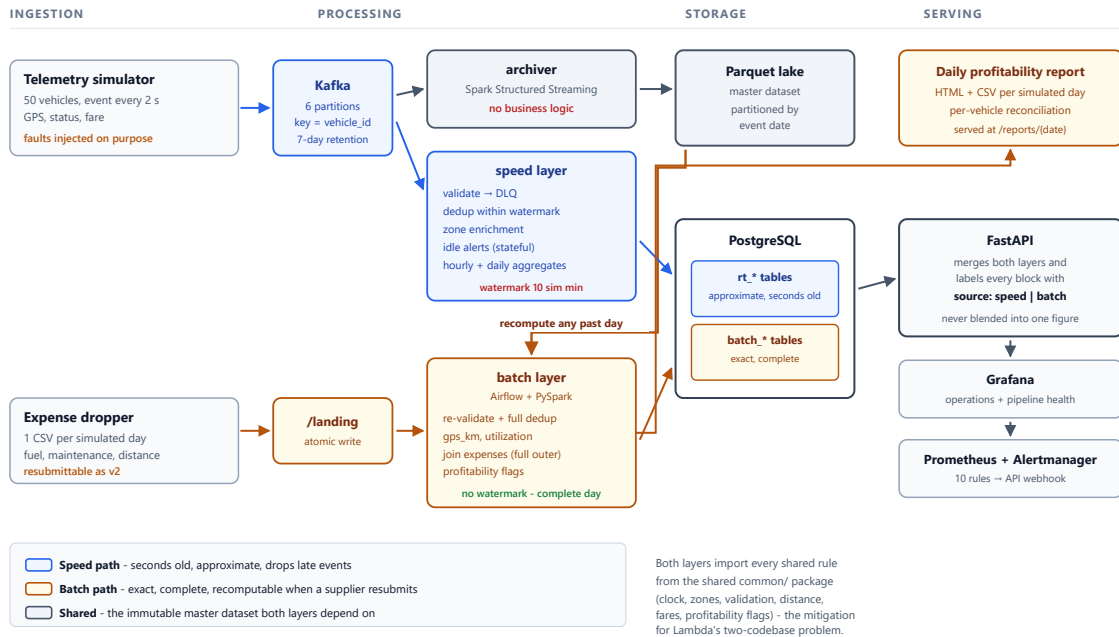


Figure 3.1: System architecture. The blue path is the speed layer (seconds old, approximate); the amber path is the batch layer (exact, complete, recomputable). Both depend on the same immutable master dataset, and both import every shared business rule from `common/`.

The decision follows from Table 1.1 rather than from a preference for a named architecture. Two consumers need different guarantees from the same facts; a single processing model must either under-serve one or lie to the other.

3.2 Why Kappa Was Rejected

Kappa proposes a single streaming layer, with recomputation achieved by replaying the event log [5]. It is the more fashionable choice, it is genuinely simpler to operate, and its critique of Lambda’s duplicated logic is correct. It therefore deserves a specific rejection rather than a dismissal.

The decisive argument is recomputation cost. Our requirement is not merely to reprocess data — it is to *restate a past day’s financial figures* when a partner sends corrected costs. Under Kappa that implies three things:

1. **Retention must cover any day we might be asked to restate.** A partner correcting last month’s file forces months of Kafka retention purely to enable periodic recomputation. Our actual retention is **7 days**, and it covers only speed-layer recovery and archiver catch-up.
2. **Replay is not selective.** Recomputing one day means re-reading the log from at least that day’s start. A Parquet lake partitioned by `sim_date` lets the batch job read exactly the one partition it needs; the measured `compute_vehicle_day` task takes **9.4 seconds** for a full simulated day.
3. **The correction is not in the stream at all.** The expense file arrives on a *different channel* — a CSV drop — once per day, long after the telemetry it relates to. There is no event log to replay that would incorporate it. Kappa’s recomputation story simply does not address the actual source of restatement.

The third point is the structural one, and it is what Section 2.5 identified as the condition under which the literature’s decision rule selects Lambda. A Kappa purist could of course publish the expense file onto a topic and treat it as a second stream. That is a real option, and we considered it; it relocates the problem rather than solving it, because restating January from a corrected file received in March still requires the January *telemetry* to be replayable, which is the retention cost of point 1 in a different guise.

A **batch-only architecture** was also considered and rejected for the obvious reason: it cannot answer “which vehicles are idle right now”. The brief requires a live utilisation endpoint and threshold alerting on idleness, neither of which survives a nightly cadence.

3.3 What Lambda Costs, and Our Mitigation

The standard criticism of Lambda is **two codebases that drift** [5]: the same business rule gets written once for streaming and once for batch, and the two copies diverge until the layers silently disagree. We accept the criticism as valid and mitigate it structurally rather than by promising discipline.

A `common/` package holds every rule both layers need — the simulated clock, zone mapping, event validation, haversine distance, fare maths and all seven profitability flags — and is imported by the simulators, the speed layer, the batch layer *and* the serving API. A business rule defined twice is treated as a defect.

Two places duplicate deliberately, both for performance, because a Python UDF executing per row over every event is the most expensive thing this pipeline could do. Table 3.1 lists them with the test that pins each pair together.

The duplication is a considered trade-off that is only defensible *because* those tests exist. Without them we would have precisely the failure mode Kreps describes.

Table 3.1: The two deliberate duplications, and the tests that make them defensible. Each test feeds identical inputs through both implementations and asserts identical outputs.

Logic	Python	Spark	Test
Event validation	<code>validate_event()</code>	<code>spark_validation_expr()</code>	<code>test_spark_validation_matches_the_python_rules</code>
Haversine distance	<code>geo.haversine_km()</code>	inline Spark SQL	<code>test_gps_km_matches_the_python_haversine</code>

3.4 Technology Stack

Table 3.2 gives each choice with the alternative it beat. Justifications are tied to this use case’s constraints, not to general popularity. Every image tag and Python package is pinned; there is no `latest` anywhere in the project.

3.5 The Simulated Clock

Everything in this project happens on a compressed timeline: **one real second is one simulated minute** (`COMPRESSION=60`). A simulated hour therefore takes one real minute and a simulated day takes 24 real minutes, which is what makes a genuinely *daily* reconciliation demonstrable inside a lab session.

The clock is derived from a persisted start instant rather than maintained as a counter, so every service computes the same `sim_time` from wall-clock elapsed time without coordinating, and a restarted container resumes on the same timeline instead of rewinding. Simulated day 1 begins at 06:00 and is therefore partial.

This is the project’s largest single simplification and it has a consequence we report rather than hide: because simulated time is derived from wall-clock time, **the pipeline being down does not pause the world**. Section 6.2 returns to this.

3.6 The Data Contract

Three fields were added to the telemetry schema given in the brief, each for a specific architectural reason.

event_id (UUID) Deduplication needs a stable identity. A duplicate is by construction *valid*, so nothing in validation can catch it.

event_type (ping/trip_start/trip_end) This one is structural. Without an explicit lifecycle marker, counting trips requires detecting a status *transition* and then aggregating, which is a chained stateful operator — and Structured Streaming forbids an aggregation after one [14]. With the marker, “trips completed” is a plain count(`WHERE event_type='trip_end'`) that works identically in both layers.

schema_version (int) Allows a future v2 producer to coexist with v1 consumers.

One interpretation of an existing field matters downstream more than any addition: fare is **cumulative within a trip and final only on trip_end**, and zero

Table 3.2: Technology selection, with the rejected alternative for each layer.

Layer	Choice	Why this, and not the alternative
Ingestion	Kafka 3.8 (KRaft)	Partitioned ordering is a <i>correctness</i> requirement here, not a throughput one (Section 4.1.1). KRaft removes the ZooKeeper dependency entirely, which halves the moving parts on a 16 GB laptop.
Speed layer	Spark Structured Streaming 3.5.3	Event-time windowing, watermarks and <code>applyInPandasWithState</code> in one engine [14]. Flink is arguably the stronger stream processor in isolation [13], but using <i>one</i> engine for both layers is precisely what makes <i>common/</i> shareable, and the shared package is our answer to Lambda’s central weakness.
Master dataset	Parquet on a volume	Columnar, partitioned by event date, read selectively [15]. MinIO was the intended object store but its community images are no longer published; every path is built from <code>LAKE_ROOT</code> , so an <code>s3a://</code> bucket is a one-variable substitution.
Batch	Airflow 2.10 + PySpark local mode	Airflow gives retries, dependency ordering and per-task observability [16]. Local mode because one simulated day is roughly 12k events, which a single JVM processes in 9 seconds; client-mode submission would add driver/executor routing problems for no measurable gain.
Serving store	PostgreSQL 16	Both layers’ views live side by side in one queryable store [17]. Upserts on natural keys give idempotent sinks (Section 4.2.3). Cassandra would suit the write-heavy <code>rt_*</code> tables but not the analytical scans over <code>batch_vehicle_daily</code> , and running both was

elsewhere. Revenue is therefore `SUM(fare) WHERE event_type='trip_end'` in both layers. Summing `fare` across all rows would multiply revenue by roughly the number of pings per trip — the single easiest way to get this pipeline’s headline number wrong — so the rule carries a test in the simulator, the speed layer and the batch layer.

Chapter 4

Implementation

4.1 Ingestion

4.1.1 Partitioning Is a Correctness Decision

The topic `fleet.telemetry` has 6 partitions, **keyed by `vehicle_id`**. This is not about throughput. Kafka guarantees ordering only *within* a partition [11], and the speed layer’s per-vehicle state machine — which tracks when each vehicle last became idle — assumes it observes one vehicle’s events in the order they occurred. Keying by `vehicle_id` provides exactly that guarantee. A round-robin key would silently corrupt every idle measurement, and would do so without producing a single error.

The dead-letter topic `fleet.telemetry.dlq` has 1 partition: it needs no ordering and carries a trickle of traffic, so one partition keeps every rejected event in a single readable stream.

4.1.2 Deliberate Fault Injection

The simulator corrupts its own output so that error handling is *demonstrable* rather than asserted. Table 4.1 gives configured against measured rates over approximately 2,700 events.

Table 4.1: Injected fault rates, configured against measured, and what each fault demonstrates.

Fault	Configured	Measured	What it proves
Malformed JSON	0.5%	0.40%	The archiver preserves unparseable bytes; the speed layer dead-letters rather than crashing
Schema-invalid	0.5%	0.77%	<code>common/validation.py</code> rejects identically in both layers
Duplicate <code>event_id</code>	1.0%	1.03%	Deduplication works; both copies are <i>valid</i> , so nothing else can stop them
Late by 1–20 sim min	2.0%	2.09%	The consistency argument — see Section 5.2

The DLQ carries the reason code *and the original payload*, so rejected events are replayable once a producer is fixed. The observed distribution across five reason codes was `unparseable_json` 58, `coords_out_of_bounds` 23, `bad_status` 17, `negative_fare` 12 and `negative_speed` 10.

Validation **rejects rather than repairs**. Coercing a malformed field would produce a confidently wrong money figure downstream, which is strictly worse than a visible rejection.

4.1.3 The Daily Batch Source

A second simulator drops one expense CSV per simulated day, carrying `vehicle_id`, `fuel_cost`, `maintenance_cost`, `distance_covered` and `service_flag`. Files are versioned: a partner correcting an earlier submission drops a `v2` file for a date already reconciled, which is the event the entire architecture exists to absorb (Section 5.3).

The service-level agreement for delivery governs **first delivery only**. An early implementation evaluated the SLA against every version, so a corrected `v2` — by definition sent after the deadline — marked every resubmission late. That is discussed as defect 5 in Section 6.1.

4.2 The Speed Layer

4.2.1 The Archiver Is Deliberately Boring

The archiver is a *separate Spark application* with its own checkpoint and **no business logic whatsoever**. The master dataset is the source of truth from which every past day is recomputed; if it is corrupted, all history is wrong and nothing can recover it. Isolating it means a bug in the speed layer’s validation, windowing or state handling *cannot reach it* — the worst such a bug can do is produce wrong dashboards, which the next batch run overwrites.

Unparseable records are archived under `sim_date=unknown` with their raw bytes intact. An archive that silently discarded malformed records would make “how much bad data did we receive?” permanently unanswerable, which is exactly the property immutability is supposed to buy [3].

4.2.2 Stateful Idle Alerts

“Has this vehicle been idle for 45 simulated minutes?” cannot be answered by a window or an aggregation. It requires memory of *when* the vehicle last stopped being idle, carried across micro-batches, and the ability to fire when **no** new event arrives — a silent vehicle being precisely the case of interest. `applyInPandasWithState` is the only Structured Streaming construct providing both [14].

Four subtleties, each a defect if mishandled:

1. **The idle stopwatch starts on the *transition* into idle**, not on every idle ping. Resetting per ping means the timer never advances and no alert can ever fire.
2. **Events are sorted by event time before folding**, because a micro-batch may interleave partitions.
3. **A stale out-of-order event does not rewind the state**. It is not lost — the archive holds it and the batch layer counts it — but rewinding would corrupt the idle measurement.
4. **EventTimeTimeout, not ProcessingTimeTimeout**. A processing-time timeout fires after N *real* seconds, so changing the clock compression would silently change the business rule.

A known consequence, stated rather than hidden: an event-time timeout only fires when the watermark advances, and the watermark only advances when other events arrive [6]. Were the entire fleet to go silent simultaneously, no timeout would fire. That case is covered by a different control — the `TelemetryNotProduced` alert, which watches the producer directly.

4.2.3 Idempotent Sinks

`foreachBatch` provides **at-least-once** delivery. If the driver dies after writing a micro-batch but before committing offsets, Spark re-runs that batch and the sink observes the same rows twice [8]. Under plain inserts, every dashboard figure would ratchet upward on each restart and never recover.

Every sink therefore upserts on a **natural key**: `rt_vehicle_state` on `(vehicle_id)`, `rt_zone_hourly` on `(zone_id, window_start)`, `rt_vehicle_daily` on `(vehicle_id, sim_date)`, and `idle_alerts` on `(vehicle_id, opened_at)`. The repeated write sets identical values, making the sink idempotent — the practical equivalent of exactly-once semantics [4].

This also permits `update` output mode for the windowed aggregations, which is necessary because `append` would withhold a window until the watermark passed its end, meaning the *current* simulated hour — the thing a live dashboard exists to show — would never appear.

4.3 The Batch Layer and Orchestration

4.3.1 The Batch Layer Trusts Nothing

`compute_vehicle_day` re-reads the raw archive and **re-applies the same validation and deduplication from scratch**. It does not read the speed layer’s output at all. Critically it applies **no watermark**: it reads a complete, closed day, so lateness is irrelevant to it, and it deduplicates across the entire day rather than within a watermark window. This independence is what makes the drift measurement in Section 5.2 meaningful — the two figures are genuinely arrived at twice.

Its outputs per vehicle-day are `trips`, `revenue`, `gps_km` (haversine between consecutive pings, with physically impossible jumps above 150 km/h discarded), `online_min`, `on_trip_min`, `idle_min` and `utilization`.

The reconciliation joins telemetry to expenses with a **full outer join**, deliberately. An inner join would silently drop the two findings the business most needs: telemetry with no expense row (`missing_costs` — we are blind to a vehicle’s costs) and an expense row with no telemetry (`missing_telemetry` — we are being billed for a vehicle that never reported).

4.3.2 The Schedule Is a Poll, Not a Cron

Airflow’s scheduler lives in real time and knows nothing about the compressed clock. The DAG therefore runs every 2 real minutes and its *first* task decides which simulated day is actually pending; the schedule is a poll and the date selection is data-driven. Expressing the schedule in simulated time was tried, and produced an interval 60 times longer than intended — a variant of defect 3 in Section 6.1.

The DAG has nine tasks: `find_pending_date`, `check_expense_file`, `validate_expenses`, `compute_vehicle_day`, `reconcile_profitability`, `load_batch_views`, `compute_drift`, `generate_report` and `record_run`.

4.3.3 Recomputation Is Delete-then-Insert

`load_batch_views` deletes every row for the target `sim_date` and inserts the recomputed set inside a single transaction. Rerunning a date therefore produces the same row count with updated values, and a partially applied recompute is impossible. This is the property the architecture exists to provide, and Section 5.3 demonstrates it on both a corrected resubmission and a forced replay.

4.4 Storage and Serving

Both layers' outputs live in one PostgreSQL instance: four `rt_*` tables written by the speed layer and four `batch_*` and bookkeeping tables written by the batch layer (Appendix B). Every `rt_*` table has a primary key on its natural key, which is what makes the streaming upserts of Section 4.2.3 possible.

4.4.1 The Lambda Merge Is Explicit, Never Blended

`GET /vehicles/{id}` returns three blocks, each labelled with its `source` and its `as_of`:

```
{ "live":    { "source": "speed", "as_of": "...", "status": "idle" },
  "today":   { "source": "speed", "trips": 4, "revenue": 512.25 },
  "history": [ { "source": "batch", "sim_date": "...", "net_profit": 570.0 } ] }
```

They are deliberately **not merged into a single figure**. The two layers answer different questions with different guarantees, and a consumer must be able to tell at a glance which numbers are safe to quote to finance. Blending them is precisely how a dashboard estimate ends up in a financial statement.

“Now” is always *simulated* now, derived from `max(last_event_time)` rather than the database clock — so if ingestion stalls, “now” stops advancing and the active-vehicle count correctly falls to zero rather than quietly lying.

The API exposes eleven documented endpoints (Appendix D), and the consolidated daily report is served as a rendered HTML document at `GET /reports/{sim_date}`. Two Grafana dashboards — *Fleet operations* and *Pipeline health* — read the same tables.

4.5 Observability

4.5.1 What Is Measured, and Why

Three questions drive every metric. Nothing is collected because it was easy to collect.

4.5.2 Structured Logging Across All Stages

Every log line is a single JSON object carrying `ts`, `level`, `service`, `stage` (`ingestion/processing/storage/serving/orchestration`), `event`, `msg` and `sim_time`.

`sim_time` is the field that matters here. On a compressed clock a wall-clock timestamp tells you nothing about *which simulated hour* a line belongs to, so every line carries both. Per-event logging is sampled 1-in-*N*: at 50 vehicles emitting every 2 seconds, an INFO line per event would bury everything else, so the useful granularity is one line per micro-batch carrying counts.

Table 4.2: The metric design. Each row answers a question that the others cannot.

Question	Metrics	Why it is not obvious
Is data arriving?	<code>fleet_events_produced_total</code> , <code>fleet_faults_injected_total</code>	Distinguishes “the producer is silent” from “the producer is gone” — the alert needs both <code>increase()==0</code> and <code>absent()</code> .
Is it being processed?	<code>fleet_stream_last_progress_timestamp</code> , <code>..._processed_rows_per_second</code> , <code>..._offsets_behind_latest</code>	Freshness is per-query, not per-process. A container can be perfectly healthy while one of its five queries is frozen.
Is it correct?	<code>fleet_dlq_events_total</code> , <code>fleet_late_rows_dropped_total</code> , <code>fleet_speed_batch_drift_ratio</code>	These measure <i>known, accepted</i> imprecision. A pipeline that cannot quantify its own error cannot be trusted with money.
Is the business healthy?	<code>fleet_open_idle_alerts</code> , the profitability flags	Deliberately separate from pipeline health: <code>IdleVehiclesHigh</code> fires when everything is working and the fleet simply is not earning.

4.5.3 Tracing — An Honest Position

We do **not** implement distributed tracing. What we implement is **correlation-ID propagation**: `event_id` travels producer → Kafka → archiver → DLQ → batch dedup; `vehicle_id` travels through every stage and is the partition key; `run_id` tags every Airflow task, `batch_runs`, `dq_issues` and `batch_vehicle_daily`; and `sim_date` identifies the lake partition, every batch table and the report filename.

A single rejected event can therefore be traced from its DLQ reason back to its original payload, and a single reconciled figure back to the run and expense-file version that produced it. What this does *not* give is per-request span timing across service boundaries. For a pipeline whose hops are Kafka offsets and Parquet partitions rather than RPC calls, spans would add infrastructure without answering a question we actually have. Section 6.3 states when that calculus flips.

4.5.4 Alerting Decisions Worth Defending

Kafka lag comes from Spark’s own query progress, not a lag exporter. Structured Streaming does not commit consumer-group offsets — it tracks them in its checkpoint — so `kafka-consumer-groups.sh` and every off-the-shelf exporter report *nothing* for these consumers. The figure exists only in the progress event’s source metrics.

Batch metrics are exported by the API from PostgreSQL, not pushed. Airflow tasks are short-lived processes; Prometheus pulls, so by scrape time the task has exited. A Pushgateway would work but is another service holding job state indef-

initely with no notion of staleness, and is explicitly discouraged for this pattern [19]. The batch layer already persists its outcomes because the API and the report need them, so re-exporting costs nothing and guarantees the metric and the report can never disagree.

Ten alert rules are defined (Appendix C), each with a `for:` duration, a severity and a runbook entry. Section 5.6 reports which have been verified firing and delivered.

Chapter 5

Results and Analysis

5.1 Evaluation Method

All figures below were obtained from the running stack, not from estimates. The system was started from a fresh clone with `make up` and left running; four simulated days were reconciled by the Airflow DAG without intervention. Metrics were read from the Prometheus endpoints and the PostgreSQL bookkeeping tables (`batch_runs`, `speed_batch_drift`, `dq_issues`, `alert_notifications`), which are the same sources the dashboards and the daily report read. Where a figure could not be measured, it is reported as not measured rather than estimated.

5.2 The Consistency Argument

This section is the empirical justification for the entire architecture.

The speed layer applies a **10-simulated-minute watermark**; the simulator injects lateness of up to **20 simulated minutes**. The gap is deliberate. A watermark of 25 would catch everything, the two layers would agree perfectly, and the project would have no evidence for why the batch layer exists. A regression test fails if anyone raises it. Figure 5.1 shows the mechanism.

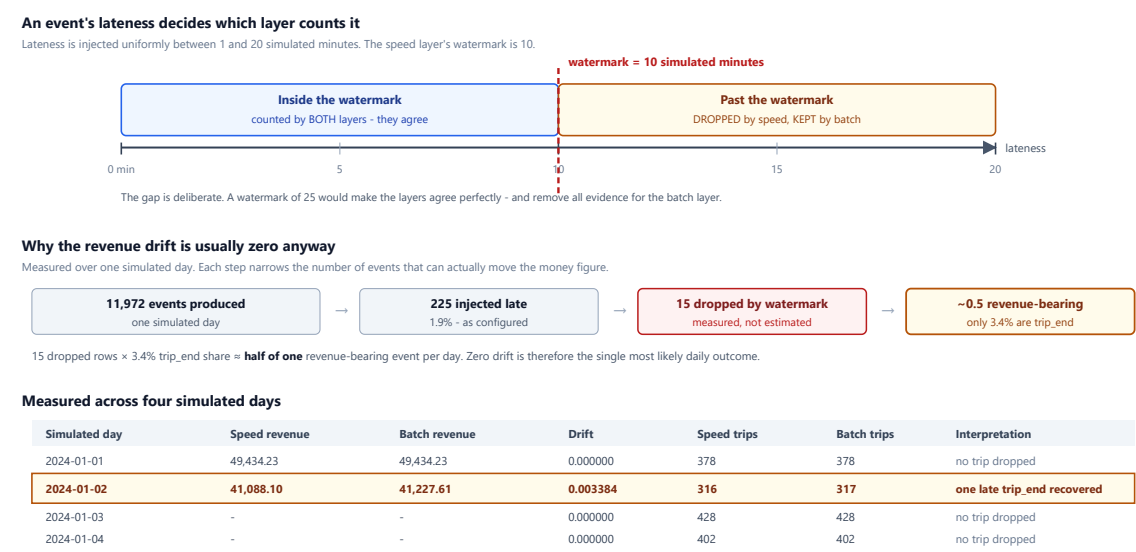


Figure 5.1: Why the two layers disagree, and by how much. An event's lateness decides which layer counts it; the funnel shows why that translates into a near-zero revenue drift on a typical day.

5.2.1 Results Over Four Simulated Days

Table 5.1: Reconciliation outcomes over four consecutive simulated days. Revenue drift is the relative difference between the speed layer’s and the batch layer’s daily revenue.

Simulated day	Fleet margin	Unprof.	Becoming unprof.	Dist. mismatch	Revenue drift
2024-01-01 (partial)	42.8%	6	0	2	0.000000
2024-01-02	38.3%	4	0	21	0.003384
2024-01-03	45.8%	3	6	6	0.000000
2024-01-04	47.7%	4	7	4	0.000000

On **2024-01-02** the drift was 0.34%, with `speed_trips` = 316 against `batch_trips` = 317: **exactly one late `trip_end` event was dropped by the watermark and recovered by the batch layer.** That single row is the clearest evidence in the project — the mechanism is real, measurable, and attributable to a specific event.

5.2.2 Why the Drift Is Usually Zero, and Why That Is the Stronger Finding

We expected a visibly non-zero drift every day. Measuring why it is usually zero produced a better argument than a dramatic number would have.

Table 5.2: The arithmetic of the watermark’s loss, measured over one simulated day.

Quantity	Measured
Events produced (one day)	11,972
Late events injected	225 (1.9%)
Rows dropped by the watermark	15
Share of events that are <code>trip_end</code>	3.4%
Expected dropped revenue-bearing events	$15 \times 0.034 \approx 0.5$ per day

The watermark drops roughly *half of one* revenue-bearing event per day. Zero drift is therefore the single most likely outcome, and non-zero appears on some days — which is precisely what Table 5.1 shows.

Two metrics, two jobs, and this report uses both. `fleet_late_rows_dropped_total` demonstrates the *mechanism* and is reliably non-zero; `fleet_speed_batch_drift_ratio` demonstrates its *financial materiality* and is usually near zero. Reporting only the second would understate the phenomenon; reporting only the first would overstate its consequence.

The case for computing money in the batch layer was never “the speed layer is wildly wrong”. It is that **the speed layer’s error is unbounded in principle and unknowable in advance.** A watermark is a bet on how late data arrives [6]; most days that bet is won comfortably. But a partner resubmitting a corrected file for last month is a restatement no watermark value addresses. Only recomputation from an immutable archive does.

We deliberately did *not* inflate the injected fault rates to manufacture a larger figure. Reporting 0.000000 alongside the arithmetic above is more defensible than reporting 8% obtained by quietly changing the simulator.

5.3 Recomputation Demonstrated

Two experiments confirm the property the architecture exists to provide.

Corrected resubmission. Dropping a v2 expense file for an already-reconciled day made it pending again. The recompute produced **the same 50 rows with different values** — net profit 19,770.78 → 21,177.99 — and the quarantined-row count fell from 1 to 0. The load is delete-then-insert per date inside a single transaction, so reruns are idempotent.

Deterministic replay. A manual trigger with `{"sim_date": "2024-01-03", "force": true}` recomputed that date to **bit-identical figures** (net 24,902.95) with a fresh `computed_at` timestamp, confirming that recomputation from the archive is deterministic, not merely repeatable.

Together these are the concrete form of the argument in Section 3.2: the correction arrived on the file channel, and the restatement was produced by re-reading one Parquet partition, with no log replay involved at any point.

5.4 Fault Handling and Data Quality

The measured fault rates in Table 4.1 track the configured rates closely, and every injected fault class was handled as designed: malformed payloads were archived with bytes intact and dead-lettered by the speed layer, schema-invalid events were rejected identically by both layers' validation, and duplicates were removed by `event_id` in both the watermark window and the full-day batch dedup.

5.5 Profitability Outcomes

Across the four reconciled days the fleet margin ranged from 38.3% to 47.7%, with 3–6 of 50 vehicles flagged unprofitable on any given day. The `becoming_unprofitable` trend flag requires three reconciled days before it will fire and correctly reported zero until that condition was met on 2024-01-03.

The daily report is generated per simulated date and served at `GET /reports/{sim_date}`; a sample is included in the repository at `docs/report-assets/`.

5.6 Alerting Verification

Ten alert rules are defined. **Five have been verified firing and delivered** end to end through Alertmanager into the `alert_notifications` table: `TelemetryNotProduced`, `ExpenseFileLate`, `DLQRateHigh`, `BatchNotRunRecently` and `ApiDown`. Delivery is demonstrable, not merely asserted.

A sixth, `BatchRunFailed`, fired but was **deliberately not delivered**. A missing partner file causes the reconciliation to record a failed run, so it fired alongside `ExpenseFileLate` for one root cause. An Alertmanager inhibit rule suppresses it, confirmed in the API with `inhibitedBy = 1`. Two alerts for one cause, where the second is misleading — nothing in the pipeline is broken, we are waiting on a supplier — is how alert channels become ignored [21].

Measured batch task durations were `compute_vehicle_day` 9.44s, `load_batch_views` 0.99s, `reconcile_profitability` 0.45s, and all others under 0.04s.

5.7 Verification Summary

Table 5.3: Verification summary. Every row was run against the live stack.

Check	Result
Unit, PySpark and API tests	202 passed
End-to-end smoke test against the live stack	28 / 28
Fresh clone → single command	All 13 services healthy, pipeline serving live data
Simulated days reconciled	4 clean days, 38–48% fleet margin
Alert rules verified firing <i>and delivered</i>	5 of 10 (a 6th fires but is intentionally inhibited)
Grafana panel queries returning live data	23 of 23

Chapter 6

Discussion

6.1 What Running It Actually Taught Us

Six defects were found by operating the system, not by reading the code. They are reported in full because each one generalises beyond this project.

1. A healthcheck that proves liveness proves nothing about work. Restarting the Spark worker killed the archiver’s executor. The driver logged `Disconnected from Spark cluster!` and then simply sat there: the query remained “active”, so `awaitTermination()` never returned. The container stayed up, **its healthcheck stayed green** — the metrics endpoint is served by the live driver — and the archiver silently stopped writing the master dataset for 26 minutes. The batch layer then reconciled a day from an incomplete archive. The fix, `streaming/watchdog.py`, monitors *progress* rather than liveness and exits so Docker restarts from the checkpoint. It was verified by deliberately killing the executors: the speed layer reported `vehicle_state has not progressed for 132s`, exited, restarted and resumed.

2. A correct pipeline can produce a useless answer. The first full reconciliation flagged **35 of 50** vehicles as unprofitable, with costs at $1.9\times$ revenue. The pipeline was correct throughout; the *economics* were not calibrated — fuel cost 7.35 CU/km against roughly 9.25 CU revenue per driven kilometre, and service visits costing $3\text{--}6\times$ a vehicle’s daily revenue. After recalibration the figure is **3–6 of 50** with a 38–48% fleet margin. Only running it end to end on realistic volumes surfaced this.

3. Intervals chosen in real seconds are $60\times$ longer in business terms. The odometer ledger checkpointed every 30 *real* seconds — which at `COMPRESSION=60` is 30 *simulated* minutes. A container restart lost roughly 90 km of ledger, and the batch layer flagged **21 of 50** vehicles for a distance mismatch the partner had not caused. A check that fires because of our own restart is worse than no check: it trains the reader to ignore it. This is the mismatch visible as the 21 distance mismatches on 2024-01-02 in Table 5.1.

4. Shared code publishes shared metrics. The batch gauges were defined in `common/metrics.py`, which every service imports — so all five services exported `fleet_batch_last_success_timestamp` at its default of 0. Prometheus saw five series, four permanently zero, and `BatchNotRunRecently` fired forever on the bogus ones. A permanently-firing false alert is worse than no alert.

5. An alert that fires on a normal operation is noise. The expense SLA was evaluated against every file version, so a corrected v2 — by definition sent after

the deadline — marked every resubmission late. The SLA now governs first delivery only.

6. Acceptance checks must actually be run. The “fresh clone → one command” criterion exposed two defects in the Windows helper script that no amount of reading would have found: PowerShell converting Docker’s normal stderr progress into a terminating error, and a wrapper function binding Docker’s `-d` flag to itself, so the stack came up in the foreground.

The common thread is that every one of these was invisible to code review and visible within minutes of operating the system and checking the numbers against expectations.

6.2 Limitations and Threats to Validity

Stated plainly, because a report claiming no weaknesses is not believed.

- **Simulated day 1 is partial** (06:00 start), so vehicles on night shifts work only part of it while carrying a full day’s standing cost. Its 42.8% margin is therefore not directly comparable with the other three days.
- **The revenue drift is usually zero** for the reasons measured in Section 5.2.2. The mechanism is visible in `fleet_late_rows_dropped_total`; its financial impact usually is not. A reader who wanted a dramatic number will not find one here.
- **becoming_unprofitable requires three reconciled days**, so it cannot be demonstrated in a short session. This is correct behaviour — we refuse to infer a trend from two points — but it constrains the demo.
- **“Lemon” vehicles are flagged more often, not always** — 25% of vehicle-days against 12.8% for the rest of the fleet. A high-maintenance vehicle only looks bad on a day it breaks, and a low-demand vehicle earns less but also spends less. Realistic, but a demo should name a specific vehicle rather than promise that every lemon appears.
- **Batch Spark runs in local mode**, so the project demonstrates distributed *streaming* but not distributed *batch*.
- **Downtime creates genuine data gaps.** The simulated clock is derived from wall-clock time, so while the stack is stopped, simulated hours pass with no events emitted. A fifth simulated day was measurably degraded by our own deliberate outage testing. The pipeline reports the reduced revenue faithfully rather than fabricating it — correct behaviour, but it means operational downtime is indistinguishable from low demand in the data.
- **One Kafka broker, RF 1.** Replication is out of scope, so broker loss means losing anything not yet archived.
- **Single-run measurements.** The four-day table is one run of one configuration. The drift figures in particular are small integer counts, so day-to-day variation is dominated by the Poisson-like arrival of late `trip_end` events rather than by anything systematic. We report the arithmetic in Table 5.2 precisely so the reader can judge the expected variance rather than over-reading four samples.

6.3 At Production Scale

Everything below is a deliberate simplification for a two-week, single-laptop project, paired with what would actually be required if this ran a real fleet.

6.3.1 Ingestion

Today: one Kafka broker, replication factor 1, 6 partitions, 7-day retention. **At scale:** a minimum of three brokers with RF 3 and `min.insync.replicas=2`. Today a broker loss means losing anything not yet archived — acceptable for a demo, indefensible for revenue data. Partition count would be driven by measured consumer throughput, but the *keying* would not change: `vehicle_id` keying is a correctness constraint, not a tuning knob, and it caps useful parallelism at the number of vehicles.

We version events with `schema_version` and validate defensively. A production deployment would put a **schema registry** (Avro or Protobuf) in front of the topic so incompatible producers are rejected at publish time rather than discovered in the DLQ. The DLQ would also gain an automated replay path; today replay is manual.

6.3.2 Storage

Today: Parquet on a Docker volume, partitioned by `sim_date`. **At scale:** object storage — already a one-variable change, since every path is built from `LAKE_ROOT`. Two additions would matter quickly. A **table format** (Iceberg or Delta Lake) would give atomic partition overwrites, schema evolution and time travel; our `load_batch_views` achieves atomicity inside PostgreSQL via delete-then-insert in a transaction, but the *lake* copy has no such guarantee, so a failed curated write can leave a partial partition. **Compaction** would become mandatory: the archiver writes every 30 seconds, a deliberate small-files trade-off that at real volume would make the batch *read* slower than the batch *compute*.

6.3.3 Processing

Today: Spark standalone, one worker, 3 cores, both streaming applications co-resident. **At scale:** Kubernetes with the Spark operator, giving per-application resource isolation and autoscaling. The single most valuable change would be **decoupling the archiver from the speed layer entirely** — they already have separate checkpoints, but they currently compete for the same worker's cores, which is how we lost 26 minutes of archiving (Section 6.1).

Batch would stop running in local mode. That choice is sound at 12k events/day and wrong at 12M; `compute_vehicle_day` is already written as ordinary Spark SQL, so the change is a submission target, not a rewrite.

6.3.4 Serving

Today: a single PostgreSQL instance serving both layers, queried directly by Grafana. **At scale:** the access patterns genuinely diverge. The `rt_*` tables are high-frequency point lookups and upserts; `batch_*` is analytical scans over history. We would keep PostgreSQL for the real-time tables, add read replicas for Grafana so dashboard queries cannot slow the streaming sinks, and move `batch_vehicle_daily` to a columnar warehouse once history outgrew a single node. The API would gain caching on `/fleet/live` — it is read far more often than the underlying data changes — and authentication, which is explicitly out of scope here.

6.3.5 Observability

Today: Prometheus, Alertmanager, Grafana, structured JSON logs, correlation IDs.

At scale: three additions, in priority order.

1. **Log aggregation** (Loki, or ELK). Today diagnosing a cross-service issue means `docker compose logs` per service. The logs are already structured JSON with a `stage` field specifically so they can be shipped and queried without reformatting — this is a deployment step, not a code change.
2. **Real distributed tracing**. As argued in Section 4.5.3, spans buy little for a Kafka-and-Parquet pipeline at this size. That calculus flips once there are multiple upstream producers and several consuming services, where “which producer caused this DLQ spike?” stops being answerable from correlation IDs alone.
3. **SLOs with error budgets**, replacing fixed thresholds [21]. `IdleVehiclesHigh > 8` is a magic number tuned to a 50-vehicle fleet; it does not survive the fleet doubling. An SLO expressed as a *ratio* would.

6.3.6 The Simulated Clock

This is the largest single simplification and its consequence is worth restating. Because simulated time is derived from wall-clock elapsed time, **the pipeline being down does not pause the world** — simulated hours keep passing with no events emitted, creating a genuine data gap. We observed exactly this: a simulated day degraded measurably during our own outage testing.

The pipeline reports the reduced revenue faithfully rather than fabricating it, which is the correct behaviour. But it means **operational downtime is indistinguishable from low demand in the data**. A production system would annotate known outage windows so that downstream consumers — and the `becoming_unprofitable` trend rule in particular — can exclude them rather than mistaking an incident for a commercial decline.

Chapter 7

Conclusions and Recommendations

7.1 Summary of Achievements

The project delivers a complete, running data platform for the ride-hailing fleet use case: two simulated ingestion paths, a Kafka topic partitioned for correctness, an immutable Parquet master dataset, a Spark Structured Streaming speed layer with stateful idle alerting, a nine-task Airflow reconciliation over PySpark, a PostgreSQL serving store holding both layers' views, a FastAPI surface of eleven endpoints, a consolidated daily profitability report, two Grafana dashboards, and ten Prometheus alert rules routed through Alertmanager. It starts from a fresh clone with one command on a 16 GB laptop, and 202 automated tests plus a 28-check end-to-end smoke test pass against it.

7.2 Key Findings

The business question posed by the brief contains two sub-questions with incompatible latency, accuracy and recomputability requirements. A Lambda architecture is justified here not by convention but by a specific property Kappa cannot supply economically: **restating a past day's financial figures when a correction arrives on a different channel, days later.**

The project demonstrates that property directly. A corrected file triggers an idempotent recompute producing the same 50 rows with updated values, and a forced replay reproduces bit-identical figures. It also *quantifies* the cost of the speed layer's approximation rather than asserting it, measuring one dropped revenue-bearing event on one day of four and explaining, arithmetically, why that is the expected magnitude. We regard the decision to report a measured drift of 0.000000 on three days, with the arithmetic that predicts it, as a stronger result than a larger number obtained by quietly raising the injected fault rate would have been.

Lambda's principal weakness — divergent codebases — is mitigated structurally by a shared `common/` package, with the two performance-motivated duplications pinned together by tests that feed identical inputs through both implementations. This is the part of the design we would defend most strongly to a reader who arrived persuaded by the Kappa critique.

7.3 Limitations

The limitations are set out in full in Section 6.2. The three that most constrain the conclusions are that batch Spark runs in local mode, so distributed batch is

designed for but not demonstrated; that the drift figures are small integer counts from a single run, so four days is not a sample from which to generalise a rate; and that the simulated clock's derivation from wall-clock time makes operational downtime indistinguishable from low demand in the resulting data.

7.4 Recommendations and Future Work

In priority order, and independent of scale:

1. **Decouple the archiver from the speed layer** onto separate workers. They already have separate checkpoints; sharing cores is what turned a worker restart into 26 minutes of lost archiving.
2. **Add a schema registry** in front of the telemetry topic, so an incompatible producer is rejected at publish time rather than discovered in the dead-letter queue, and give the DLQ an automated replay path.
3. **Adopt a lake table format** (Iceberg or Delta Lake) to give the curated lake writes the same atomicity that delete-then-insert already gives the PostgreSQL views.
4. **Annotate outage windows** on the simulated timeline, so the `becoming_unprofitable` trend rule can exclude days degraded by pipeline downtime instead of reading them as a commercial decline.
5. **Express alert thresholds as ratios** rather than absolute counts, so that `IdleVehiclesHigh` survives a change in fleet size.

7.5 Lessons Learned

The most valuable outcome was not the architecture but what operating it revealed: a green healthcheck over a dead archiver, a correct pipeline producing economically meaningless output, and an interval that was $60\times$ longer than intended because it was expressed in the wrong clock. None of the three was visible in code review. Each was found by running the system and checking the numbers against expectations, which is the habit this project is ultimately arguing for — and the same habit that produced its central result, since the decision to measure why the drift was zero, rather than to assume the measurement was broken, is what turned an anticlimax into the report's strongest section.

References

- [1] Department of Electrical and Information Engineering, “EC8203 applied big data engineering: Data engineering mini-project assessment.” Faculty of Engineering, University of Ruhuna, 2026.
- [2] N. Marz and J. Warren, *Big Data: Principles and Best Practices of Scalable Real-Time Data Systems*. Shelter Island, NY: Manning Publications, 2015.
- [3] P. Helland, “Immutability changes everything,” in *Proceedings of the 7th Biennial Conference on Innovative Data Systems Research (CIDR)*, 2015.
- [4] M. Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. Sebastopol, CA: O’Reilly Media, 2017.
- [5] J. Kreps, “Questioning the lambda architecture.” O’Reilly Radar, 2014. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>. Accessed 2026-09-27.
- [6] T. Akidau, R. Bradshaw, C. Chambers, S. Chernyak, R. J. Fernández-Moctezuma, R. Lax, S. McVeety, D. Mills, F. Perry, E. Schmidt, and S. Whittle, “The dataflow model: A practical approach to balancing correctness, latency, and cost in massive-scale, unbounded, out-of-order data processing,” *Proceedings of the VLDB Endowment*, vol. 8, no. 12, 2015.
- [7] T. Akidau, A. Balikov, K. Bekiroğlu, S. Chernyak, J. Haberman, R. Lax, S. McVeety, D. Mills, P. Nordstrom, and S. Whittle, “MillWheel: Fault-tolerant stream processing at internet scale,” *Proceedings of the VLDB Endowment*, vol. 6, no. 11, 2013.
- [8] M. Armbrust, T. Das, J. Torres, B. Yavuz, S. Zhu, R. Xin, A. Ghodsi, I. Stoica, and M. Zaharia, “Structured streaming: A declarative API for real-time applications in Apache Spark,” in *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018.
- [9] M. Zaharia, T. Das, H. Li, T. Hunter, S. Shenker, and I. Stoica, “Discretized streams: Fault-tolerant streaming computation at scale,” in *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, 2013.
- [10] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2012.

- [11] J. Kreps, N. Narkhede, and J. Rao, “Kafka: A distributed messaging system for log processing,” in *Proceedings of the 6th International Workshop on Networking Meets Databases (NetDB)*, 2011.
- [12] Apache Software Foundation, “Apache Kafka documentation.” <https://kafka.apache.org/documentation/>, 2024. Accessed 2026-09-27.
- [13] P. Carbone, A. Katsifodimos, S. Ewen, V. Markl, S. Haridi, and K. Tzoumas, “Apache Flink: Stream and batch processing in a single engine,” *IEEE Data Engineering Bulletin*, vol. 38, no. 4, 2015.
- [14] Apache Software Foundation, “Structured streaming programming guide, Apache Spark 3.5.3.” <https://spark.apache.org/docs/3.5.3/structured-streaming-programming-guide.html>, 2024. Accessed 2026-09-27.
- [15] Apache Software Foundation, “Apache Parquet file format specification.” <https://parquet.apache.org/docs/>, 2024. Accessed 2026-09-27.
- [16] Apache Software Foundation, “Apache Airflow documentation, version 2.10.” <https://airflow.apache.org/docs/>, 2024. Accessed 2026-09-27.
- [17] The PostgreSQL Global Development Group, “PostgreSQL 16 documentation.” <https://www.postgresql.org/docs/16/>, 2024. Accessed 2026-09-27.
- [18] S. Ramírez, “FastAPI documentation.” <https://fastapi.tiangolo.com/>, 2024. Accessed 2026-09-27.
- [19] Prometheus Authors, “Prometheus documentation: Querying, alerting rules and the pushgateway.” <https://prometheus.io/docs/>, 2024. Accessed 2026-09-27. The Pushgateway guidance cited in Section 4.5 is from *When To Use The Pushgateway*.
- [20] Grafana Labs, “Grafana documentation.” <https://grafana.com/docs/grafana/latest/>, 2024. Accessed 2026-09-27.
- [21] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, eds., *Site Reliability Engineering: How Google Runs Production Systems*. Sebastopol, CA: O’Reilly Media, 2016.

Appendix A

Repository Layout

Table A.1: Top-level layout of the submitted repository.

Directory	Contents
<code>common/</code>	Shared by <i>both</i> layers: <code>simclock</code> , <code>geo</code> (zones and haversine), <code>schemas</code> , <code>validation</code> , <code>earnings</code> , <code>profitability</code> , JSON logging, <code>metrics</code> , <code>db upsert helpers</code> , <code>config</code>
<code>simulators/</code>	<code>telemetry/</code> (Kafka producer and fleet state machine), <code>expenses/</code> (daily CSV dropper), <code>control.py</code>
<code>streaming/</code>	<code>archiver.py</code> , <code>speed.py</code> , <code>state.py</code> , <code>sinks.py</code> , <code>listener.py</code> , <code>watchdog.py</code>
<code>batch/</code>	<code>runs.py</code> , <code>expenses.py</code> , <code>vehicle_day.py</code> , <code>reconcile.py</code> , <code>report.py</code>
<code>airflow/dags/</code>	<code>fleet_daily_reconciliation.py</code> (9 tasks)
<code>api/</code>	FastAPI: <code>app.py</code> , <code>queries.py</code> , <code>models.py</code> , <code>batch_metrics.py</code>
<code>db/init/</code>	PostgreSQL schema, indexes and views
<code>observability/</code>	<code>prometheus/</code> (config and 10 alert rules), <code>alertmanager/</code> , <code>grafana/</code> (provisioning and 2 dashboards)
<code>scripts/</code>	<code>simclock_init</code> , <code>create_topics</code> , <code>wait_for_stack</code> , <code>demo</code> , <code>smoke_test</code> , <code>report build helpers</code>
<code>tests/</code>	202 tests: unit, PySpark and API
<code>docs/</code>	<code>SPEC.md</code> , <code>this report</code> , <code>decisions.md</code> , <code>contributions.md</code> , <code>runbook.md</code> , <code>walkthrough.md</code> , <code>report-assets/</code>

Appendix B

Database Schema

Every `rt_*` table carries a primary key on its *natural* key, which is what makes the streaming upserts of Section 4.2.3 idempotent.

Table B.1: Serving-store tables, by owning layer.

Table	Key	Purpose
<i>Speed layer</i>		
<code>rt_vehicle_state</code>	<code>(vehicle_id)</code>	Current status, zone and last event time per vehicle
<code>rt_zone_hourly</code>	<code>(zone_id, window_start)</code>	Windowed trips and earnings by zone and simulated hour
<code>rt_vehicle_daily</code>	<code>(vehicle_id, sim_date)</code>	Running daily totals from the speed layer
<code>idle_alerts</code>	<code>(vehicle_id, opened_at)</code>	Threshold alerts raised by the stateful idle machine
<i>Batch layer and bookkeeping</i>		
<code>batch_vehicle_daily</code>	<code>(vehicle_id, sim_date)</code>	Reconciled per-vehicle-day profitability and its seven flags
<code>batch_runs</code>	<code>(run_id)</code>	One row per reconciliation attempt, with status and expense version
<code>dq_issues</code>	<code>(id)</code>	Quarantined rows with reason, by date
<code>speed_batch_drift</code>	<code>(sim_date)</code>	Per-date speed-versus-batch revenue comparison
<code>alert_notifications</code>	<code>(id)</code>	Alerts delivered by Alertmanager to the API webhook

Appendix C

Alert Rule Inventory

Table C.1: The ten Prometheus alert rules. *Verified* means observed firing **and** delivered into `alert_notifications`.

Alert	Stage	Sev.	Condition	Verified
TelemetryNotProduced	ingestion	critical	No events produced for over a minute	yes
DLQRateHigh	ingestion	warning	More than 2% of events dead-lettered	yes
StreamStalled	processing	critical	A named query has stopped progressing	–
StreamLagHigh	processing	warning	Query more than 5,000 off-sets behind	–
ExpenseFileLate	orchestration	warning	An expense file missed its first-delivery SLA	yes
BatchRunFailed	orchestration	critical	Last reconciliation run failed	inhibited
BatchNotRunRecently	orchestration	warning	No successful reconciliation in 45 minutes	yes
SpeedBatchDriftHigh	processing	warning	Speed and batch revenue differ by over 5%	–
ApiDown	serving	critical	The serving API is not reachable	yes
IdleVehiclesHigh	serving	warning	More than 8 vehicles idle past threshold	–

Each rule carries a `for:` duration of 1–2 real minutes. The durations are short on purpose: at `COMPRESSION=60`, one real minute is one simulated hour, so a conventional five-minute `for:` would mean waiting five simulated hours before an alert fired. Every rule has a corresponding entry in `docs/runbook.md` giving what it means, how to reproduce it and how to fix it.

Appendix D

API Surface

Table D.1: The eleven documented endpoints, by the layer that answers them.

Endpoint	Layer	Answers
GET /health	ops	Liveness, plus <code>sim_time</code> and clock compression
GET /fleet/live	speed	Active vehicles, idle ratio, trips/hour, earnings
GET /zones/live	speed	Current utilisation and earnings by zone
GET /zones/{id}/hourly	speed	One zone's windowed history by simulated hour
GET /alerts/idle	speed	Open and closed idle alerts
GET /vehicles/unprofitable	batch	Vehicles flagged unprofitable or becoming unprofitable
GET /vehicles/{id}	both	The explicit Lambda merge: <code>live</code> and <code>today</code> from speed, <code>history</code> from batch, each labelled
GET /reports/{sim_date}	batch	The consolidated daily reconciliation report (HTML)
GET /pipeline/status	ops	Reconciled dates, last run, drift, expense-file state
POST /alerts/webhook	ops	Alertmanager delivery target; writes <code>alert_notifications</code>
GET /metrics	ops	Prometheus exposition, including re-exported batch metrics

Appendix E

Statement of Individual Contributions

The project was delivered by a three-member group. The work was divided by **pipeline stage**, so that each member owned one end-to-end vertical slice — its source code, its schema objects, its tests and its sections of this report — rather than all three editing the same files. Table E.1 records the principal areas each member led. Design decisions were taken jointly and code was reviewed across the team.

Table E.1: Principal areas of responsibility by team member.

Member	Reg. No.	Principal areas
Ahamed N.F.F.	EG/2021/4390	Ingestion and the data contract: <code>simulators/</code> (telemetry producer, fleet state machine, fault injection, expense dropper), Kafka topic setup and partitioning, and the <code>simclock</code> , <code>schemas</code> , <code>validation</code> , <code>geo</code> and <code>earnings</code> modules of <code>common/</code>
Safeer S.M.	EG/2021/4771	Stream processing and serving: <code>streaming/</code> (archiver, speed layer, per-vehicle state machine, sinks, listener, watchdog), <code>api/</code> , the <code>rt_*</code> tables, and the db upsert helpers in <code>common/</code>
Nafla M.N.P.	EG/2021/4687	Batch, orchestration and observability: <code>batch/</code> , the <code>fleet_daily_reconciliation</code> DAG, all of <code>observability/</code> (Prometheus rules, Alertmanager, both Grafana dashboards), and the <code>profitability</code> , <code>metrics</code> and <code>logging</code> modules of <code>common/</code>

Decisions each member defends

Ahamed N.F.F. Partitioning by `vehicle_id` as an ordering requirement (Section 4.1.1); the $60\times$ simulated clock and its derivation from a persisted start instant (Section 3.5); the deliberate fault rates (Section 4.1.2); and the decision that validation rejects rather than repairs.

Safeer S.M. The deliberately boring archiver (Section 4.2.1); the four subtleties of the stateful idle machine and the choice of `EventTimeTimeout` (Section 4.2.2); idempotent sinks and their natural keys (Section 4.2.3); and the explicit, never-blended Lambda merge (Section 4.4.1).

Nafla M.N.P. The batch layer trusting nothing and applying no watermark (Section 4.3.1); delete-then-insert recomputation (Section 5.3); the poll-not-cron DAG schedule; and the alerting design, including the inhibition rule (Section 4.5.4).

Deliberately shared: the `common/` package

`common/` is the project's answer to Lambda's central weakness (Section 3.3). Zone mapping, distance, fare, validation and profitability rules are defined *once* and imported by the simulator, both processing layers and the API. Each module has a primary author, listed above, but the rule the team worked to was that no business rule is defined twice, and a change to a `common/` module is reviewed by whichever members' layers import it. That constraint is why every member can explain modules they did not personally write: their own layer depends on them.

Worked on jointly

`docs/SPEC.md`, written together before any code and treated as fixed thereafter; `docker-compose.yml`, the `Makefile` and `common/config.py`, since every member's service is defined in them; the smoke test, to which each member contributed the checks for their own layer; the architecture argument of Chapter 3 and the consistency analysis of Section 5.2; and every acceptance run — the fresh-clone start, the multi-day reconciliation and the alert-firing verification each require all three layers alive at once.

A fuller per-member breakdown, including the test files each member wrote, is kept in `docs/contributions.md` in the repository.